Mastering Jetpack Compose

Animation, Graphics, and Architecture

December 03, 2025

Visibility Animation

Bringing UI to life with The AnimatedVisibility

Composable

Core Concepts

AnimatedVisibility

The simplest way to animate the appearance and disappearance of content. It observes a Boolean state parameter (e.g., visible = true) and automatically triggers enter and exit transitions when that state changes.

Animation Effects

Instead of instant toggling, you can apply smooth effects.

- **Fade**: Opacity transition.
- **Slide**: Movement from an offset.
- **Expand/Shrink**: Clipping bounds change.

Customizing Animations

Enter & Exit

Combine multiple effects using the + operator. For example: enter = fadeIn() + expandVertically() creates a sophisticated entrance.

Specs

Control the physics and timing using animationSpec. Use tween() for duration or spring() for bouncy, physical feedback.

Scope

Use the animateEnterExit modifier within the scope to apply specific animations to individual child composables.

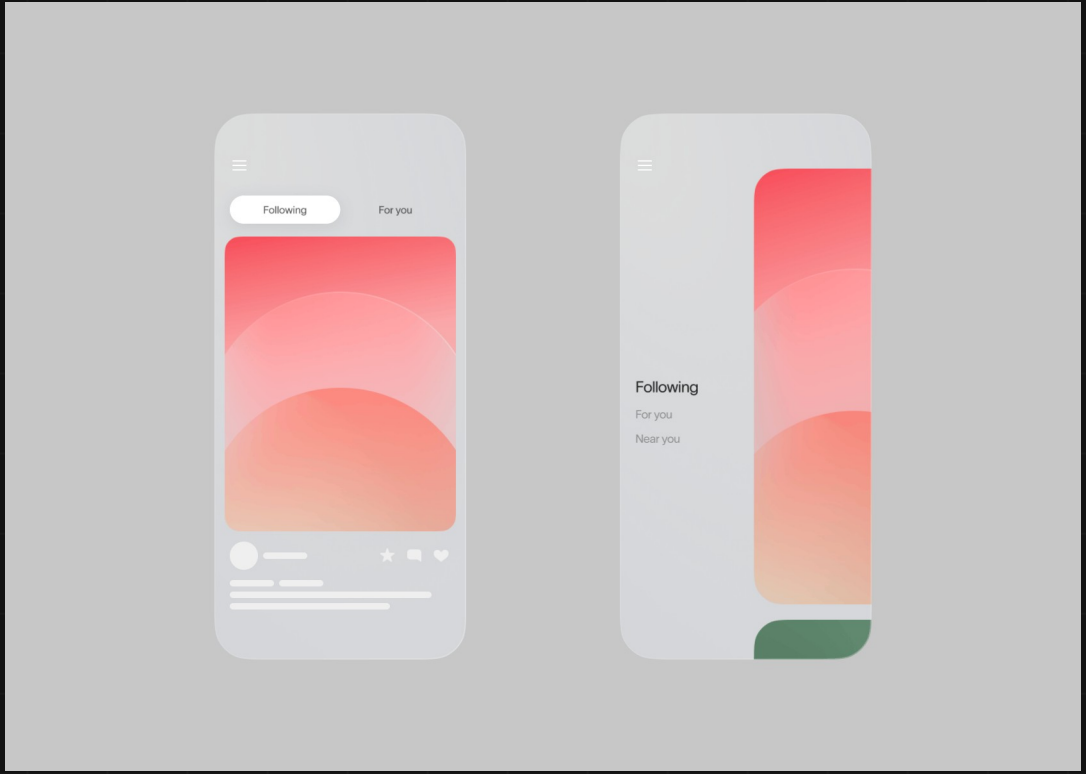
Transitions & Auto-Start

Crossfade

Ideally suited for switching between two composables (e.g., Loading vs Content). It fades out the old layout while fading in the new one seamlessly.

Auto-Starting

Use MutableTransitionState to trigger animations immediately upon composition. By setting the initialState different from the targetState, the animation begins as soon as the composable enters the screen.



State-Driven Animation

Animating Properties over

Time

The animate*AsState Family

Single Value Animation

Compose provides a suite of functions for simple "fire-and-forget" animations.

- animateFloatAsState
- animateColorAsState
- animateDpAsState

These handle the heavy lifting of interpolation for you.

Interpolation Logic

When the target value changes, the composable automatically animates from the current value to the new target. This is reactive and declarative—you define the "what" (target), and Compose handles the "how" (frames).

Physics & Timing Control

- **Tween:** Standard duration-based animation. You can define the duration in milliseconds and apply easing curves like LinearOutSlowInEasing.
- **Spring:** Physics-based animation that mimics real-world forces. It uses DampingRatio (bounciness) and Stiffness (speed) instead of duration.
- **Keyframes:** precise control over the animation timeline, allowing you to define specific values at specific timestamps (e.g., 100.dp.at(500)).
- **updateTransition:** A powerful API for coordinating multiple animations simultaneously based on a single state change, ensuring synchronization.

Canvas Graphics

Custom Drawing and Geometry in Compose

The Canvas Composable

Accessing DrawScope

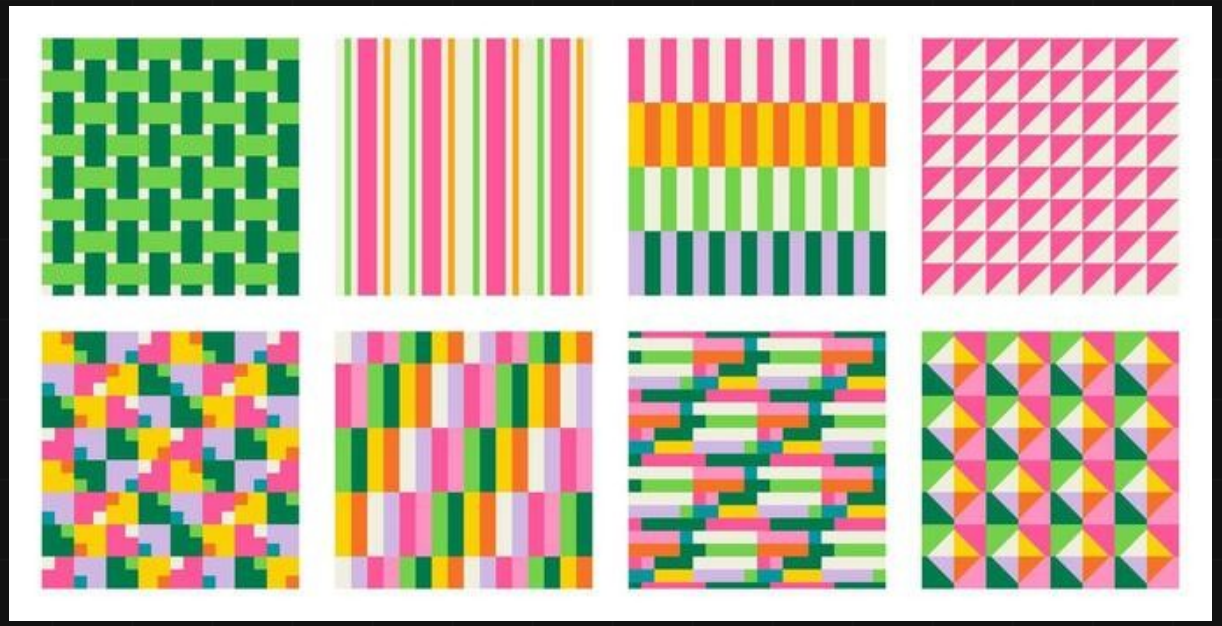
The Canvas composable provides a DrawScope, giving you a declarative surface for 2D graphics. It manages state and provides access to the drawing environment's bounds.

Primitives

Drawing is intuitive with simple function calls:

- drawLine
- drawRect / drawRoundRect
- drawCircle

The coordinate system starts at (0,0) in the top-left corner.



Advanced Drawing

Path & Curves

Create complex custom shapes using the Path object. Use commands like moveTo, lineTo, and quadraticBezierTo to craft intricate geometries.

Gradients

Fill shapes with rich colors using Brush. Compose supports Linear, Radial, and Sweep gradients with configurable color stops and tile modes.

Text & Images

Render bitmaps using drawImage. For typography, use TextMeasurer within the canvas to calculate layout and render text manually.

Architecture

ViewModels, Jetpack, and Separation of Concerns

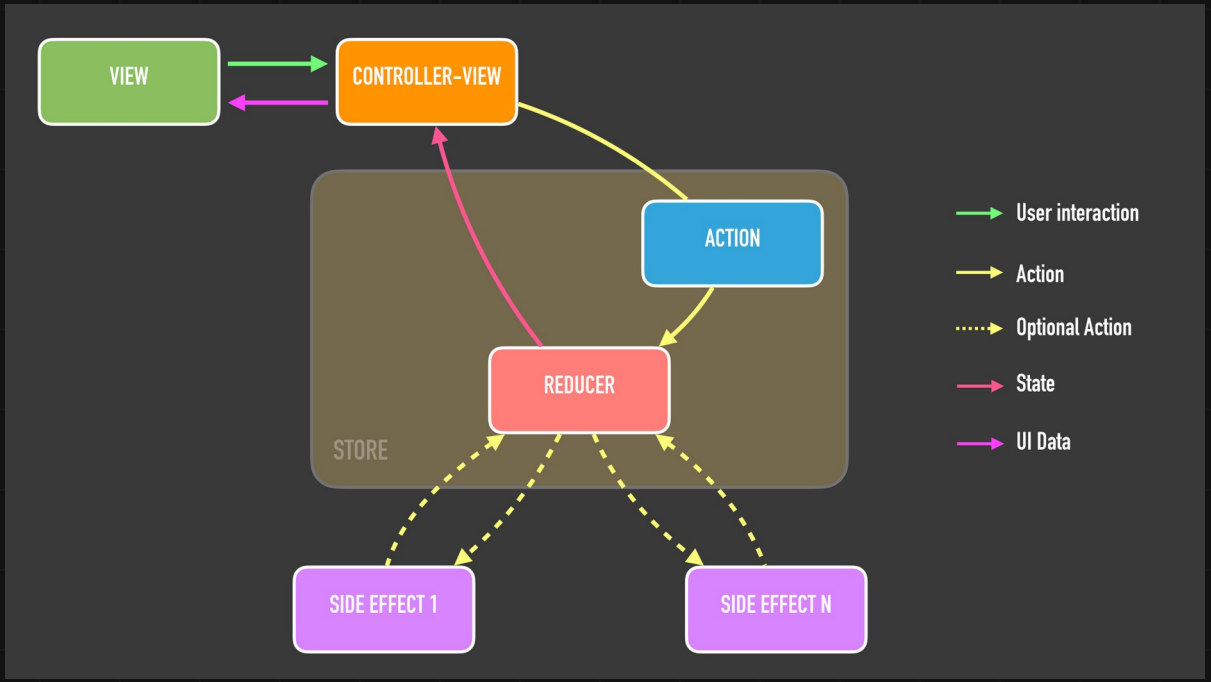
Modern App Architecture

Separation of Concerns

Modern Android development advocates for a Single Activity model where the UI is separated from the business logic. This is where **Jetpack** components shine.

The ViewModel

The ViewModel is responsible for holding state and logic. It survives configuration changes (like screen rotation), ensuring data persists while the UI (Activity/Composable) is recreated.



Implementation Details

Setup & Injection

Define your model by extending the ViewModel class. In your composables, access it easily using the viewModel() function (requires the lifecycle-viewmodel-compose dependency).

Observability

Store your data in the ViewModel using mutableStateOf for a Compose-native approach.

Alternatively, if using LiveData, convert it in the UI using observeAsState() to trigger recomposition on updates.

Questions

Thank you for your attention.